EntitySpaces 2026 – Quick Start Guide

For developers who want to get started in less than 15 minutes.

Last updated: July 30, 2026

📑 Table of Contents

- EntitySpaces 2026 – Quick Start Guide
  - 📑 Table of Contents
  - 🚀 What's New in EntitySpaces Studio 2026
  - 1. Requirements
  - 2. Step-by-Step Installation
    - 2.1. Download the Installer
    - 2.2. Run the Installer
    - 2.3. Welcome Screen
    - 2.4. Select the Installation Folder
    - 2.5. Installation Confirmation
    - 2.6. Installation Progress
    - 2.7. Installation Complete
  - 3. First Launch and Configuration
    - 3.1. Access the Studio
    - 3.2. Verify the Installed Version
  - 4. Connect to Your Database
  - 5. Generate C# or VB.NET Classes
    - 5.1. Open the Templates Panel
    - 5.2. Select the "Generated - Classes Master" Template
    - 5.3. Configure Generation Parameters
    - 5.4. Run the Generation
    - 5.5. Verify the Generated Code
  - 6. Generate Custom Classes (Generated Once)
    - 6.1. Run the "Custom - Classes Master" Template
    - 6.2. Review the Generated Structure
    - 6.3. Explore the Custom Class Code
  - 7. Folder Structure in Visual Studio
  - 8. Unit Testing with EntitySpaces and PostgreSQL
    - 8.1. Configure the Loader and Connection in Your Test
    - 8.2. Load and Query Tests

        - 8.3. Run the Tests
    - 9. Add NuGet Packages to Your Project
    - 10. Configure the Loader and Connection in Your Code
    - 11. Write Your First Query
    - 12. Using with Wisej.NET
    - 13. Common Troubleshooting
    - 14. Additional Resources
        - Ready! With these steps, your environment is up and running, and you can start developing with EntitySpaces 2026.

🚀 What's New in EntitySpaces Studio 2026

This version of the Studio includes important maintenance and performance improvements:

- **Metadata Engine Modernization** – Advanced schema properties are now extracted (`CharacterMaxLength`, `NumericPrecision/Scale`, `IsComputed`, `IsConcurrency`, `IsAutoIncrement`, `HasDefault` and `DataTypeNameComplete`) for all providers (SQL Server, PostgreSQL, MySQL, SQLite, Oracle and Firebird). The generated code accurately reflects your database structure.
- **Fixed x86 Architecture** – The `StandAlone.exe` executable is always compiled for x86, ensuring compatibility with native assemblies (`SQLite.Interop.dll`, ODP.NET). No more architecture errors when opening connections.

---

1. Requirements

- **.NET Framework 4.8** or **.NET 8/9/10** (depending on your project).
- **Visual Studio 2022** or later with Desktop/Web development workloads.
- **EntitySpaces Studio 2026** (standalone code generator).
- Administrator privileges for installation (optional but recommended).

---

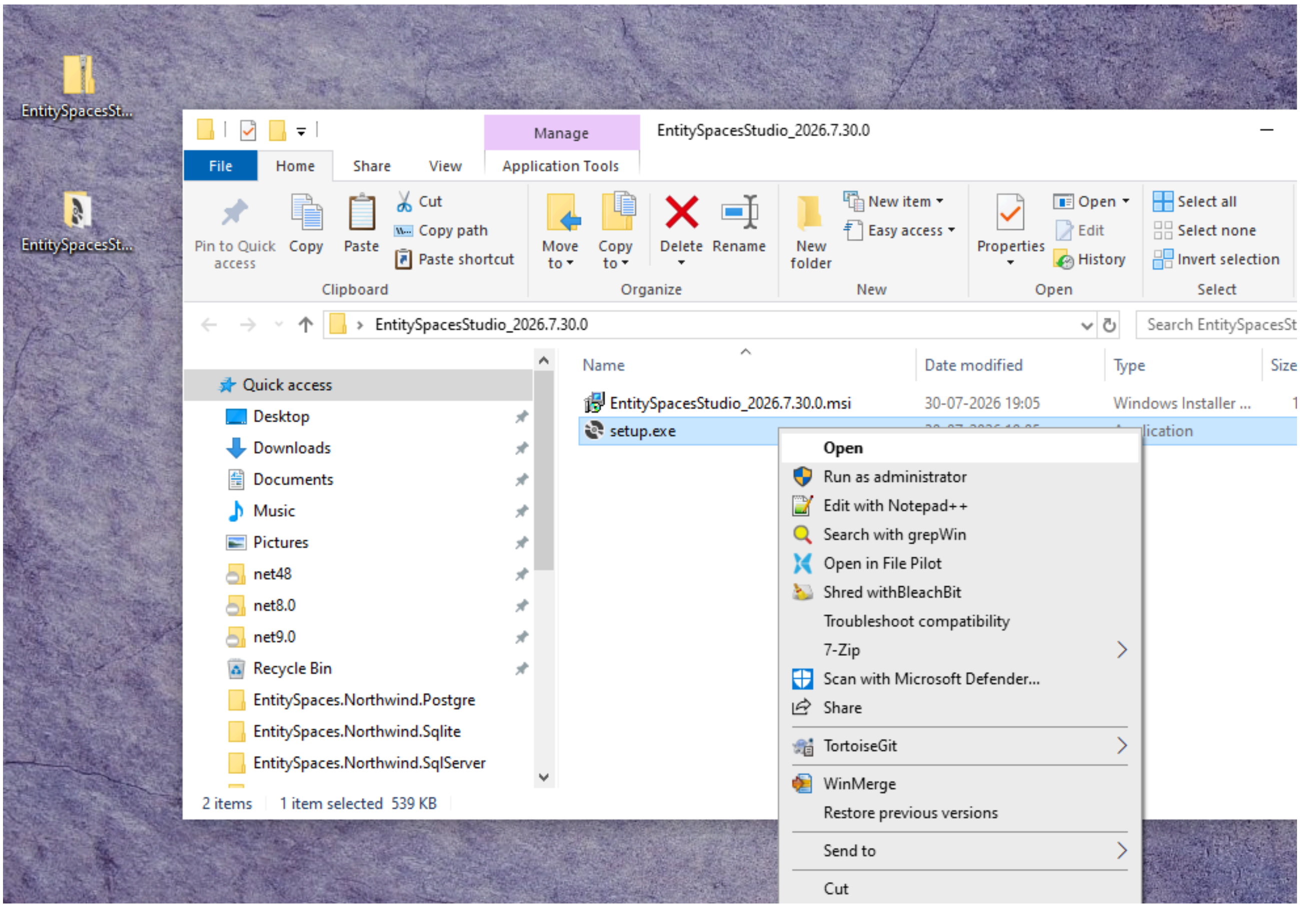
2. Step-by-Step Installation

2.1. Download the Installer

1. Go to the GitHub repository:
   https://github.com/paulcordova/EntitySpaces
   and download the `EntitySpacesStudio_2026.7.0030.0.exe` file (or the latest version) from the **Releases** section.

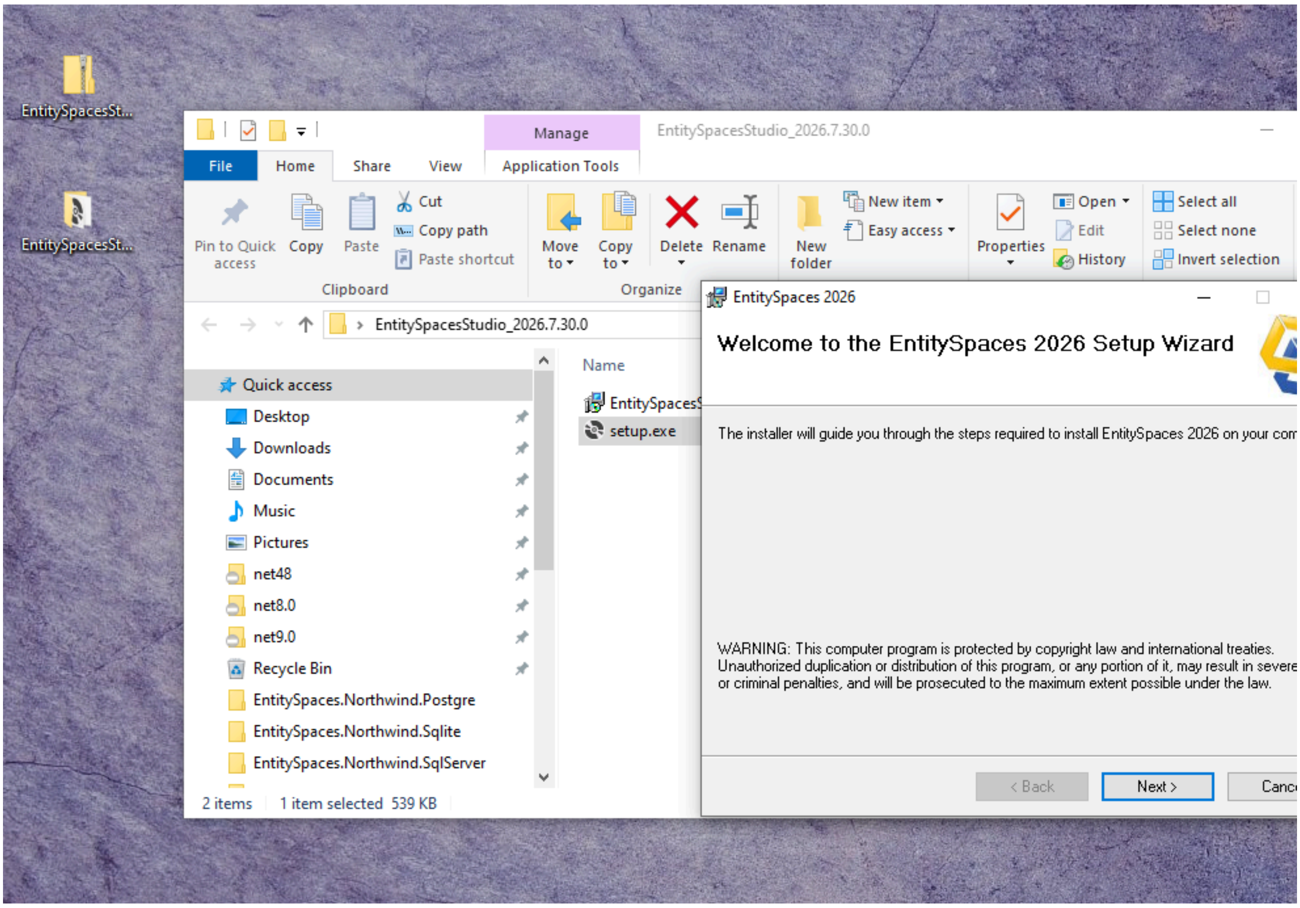
2. Save the file to your computer (e.g., `Downloads`).

2.2. Run the Installer



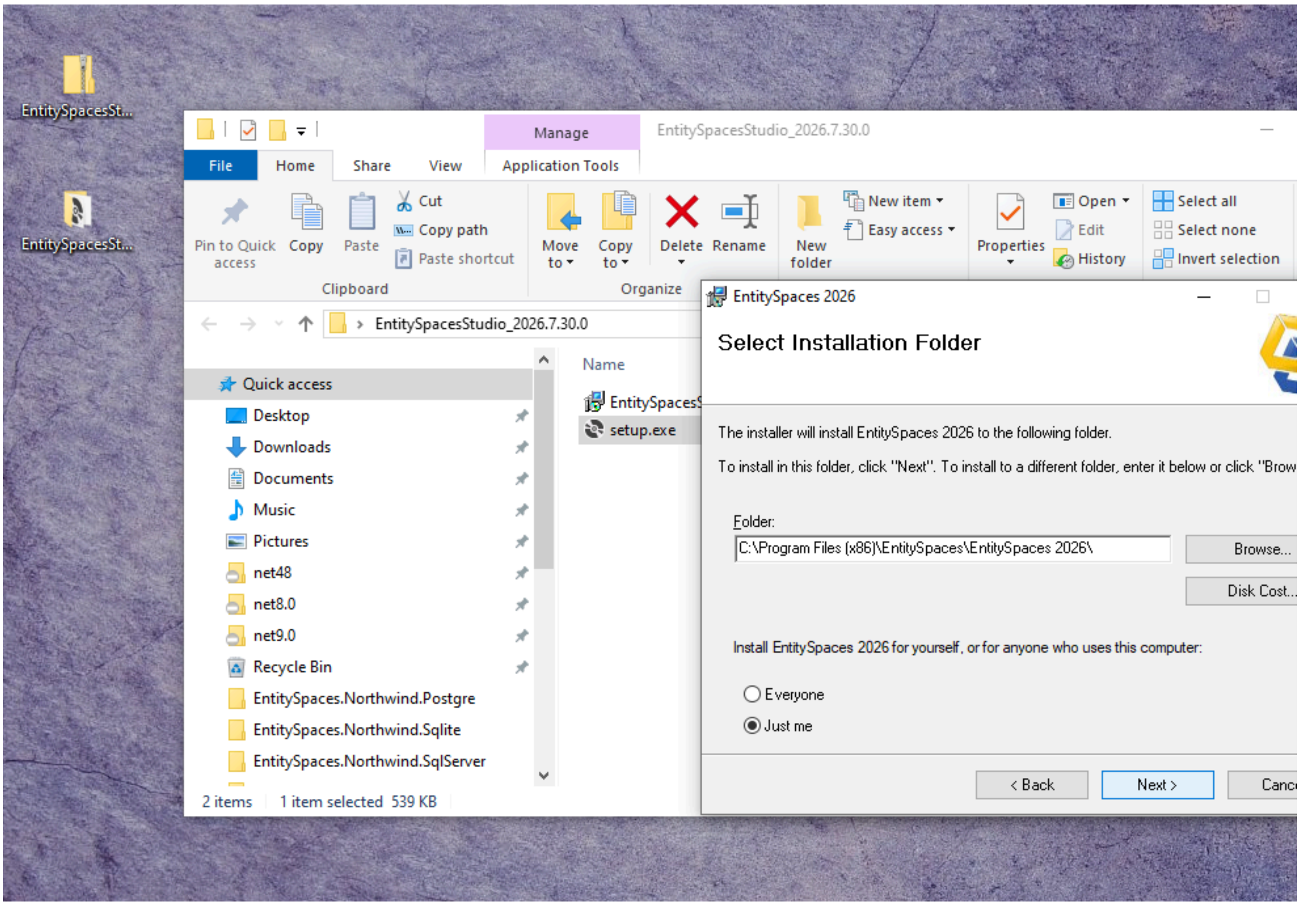
Double-click the downloaded `.exe` file. If Windows shows a security warning, click **"Run anyway"**.

2.3. Welcome Screen



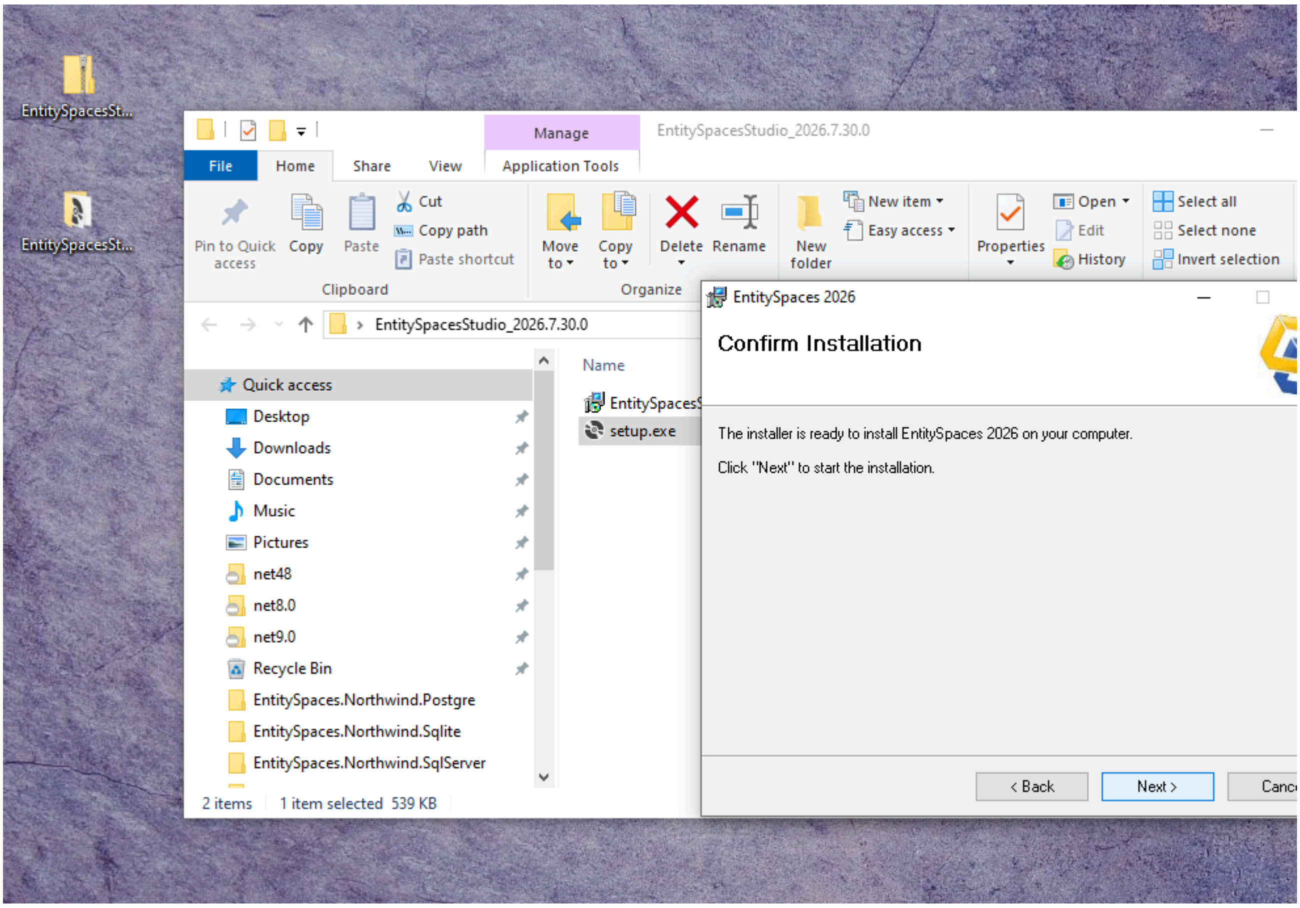
The installation wizard will open. Click **"Next"** to continue.

2.4. Select the Installation Folder



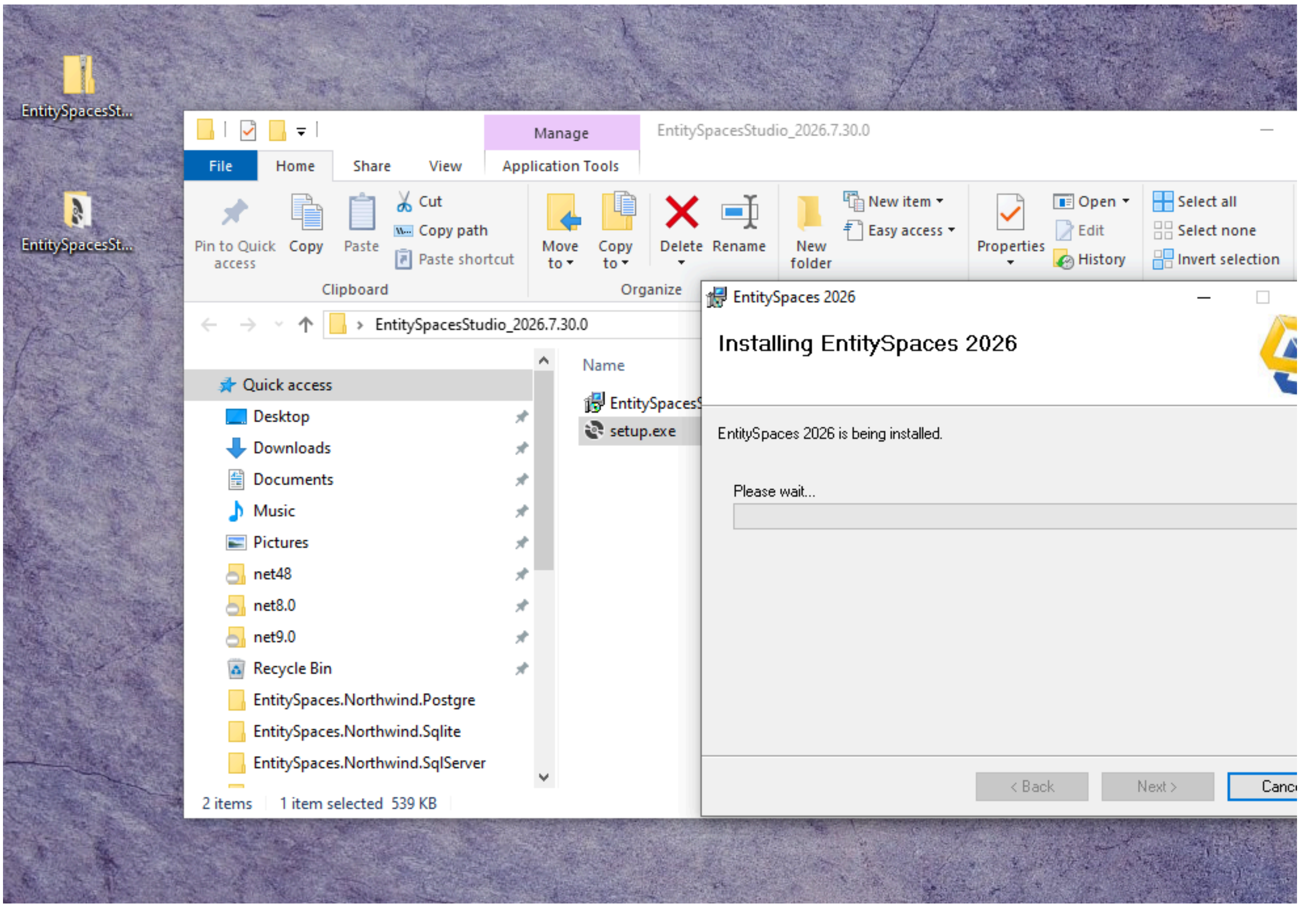
By default, the installer suggests `C:\Program Files (x86)\EntitySpaces 2026\`. You can change it if you wish. Click **"Next"** after choosing the location.

2.5. Installation Confirmation



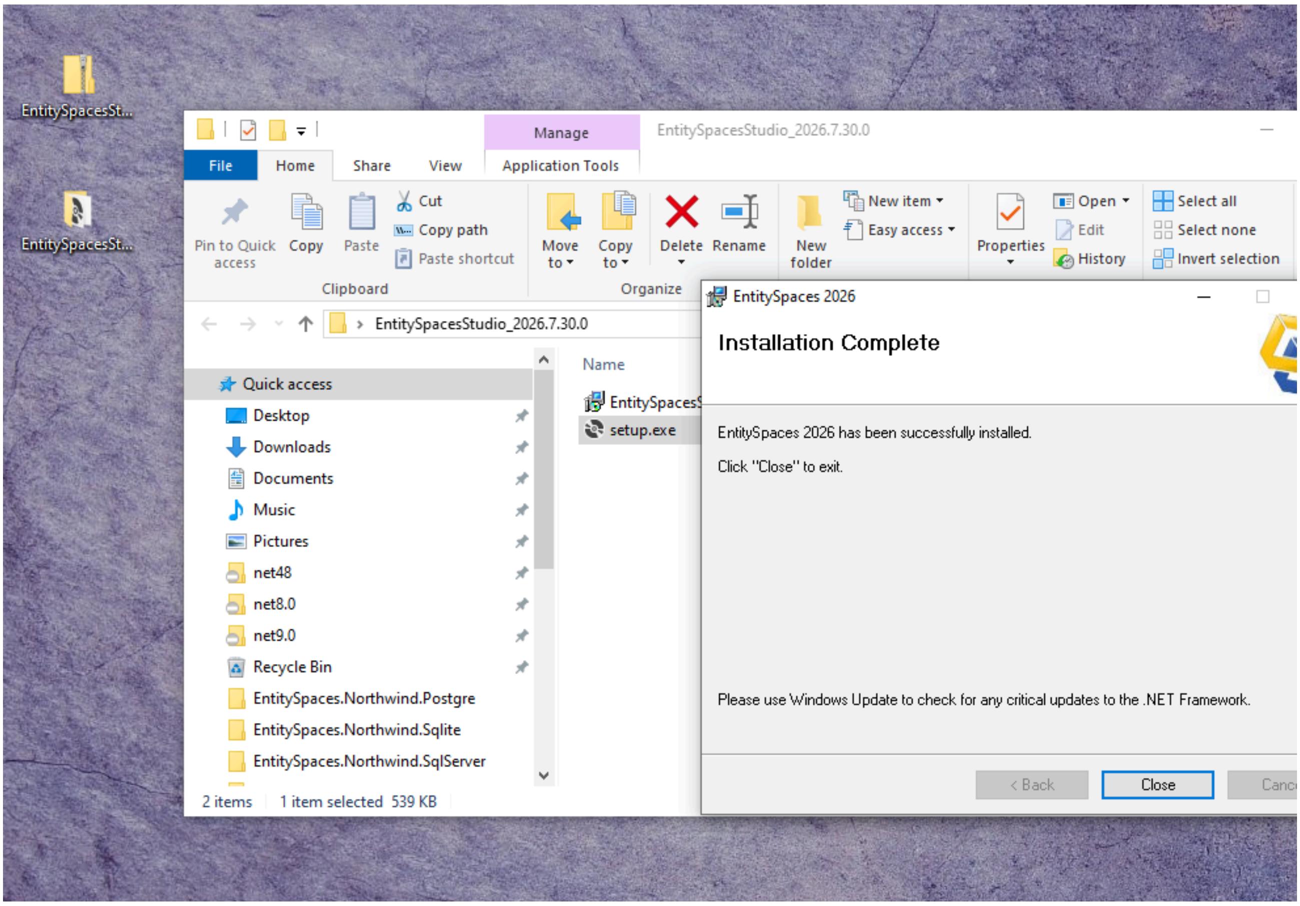
Review the destination folder and the components to be installed. Click **"Install"** to begin copying files.

2.6. Installation Progress



Wait while the installer copies the files and registers the necessary components.

2.7. Installation Complete

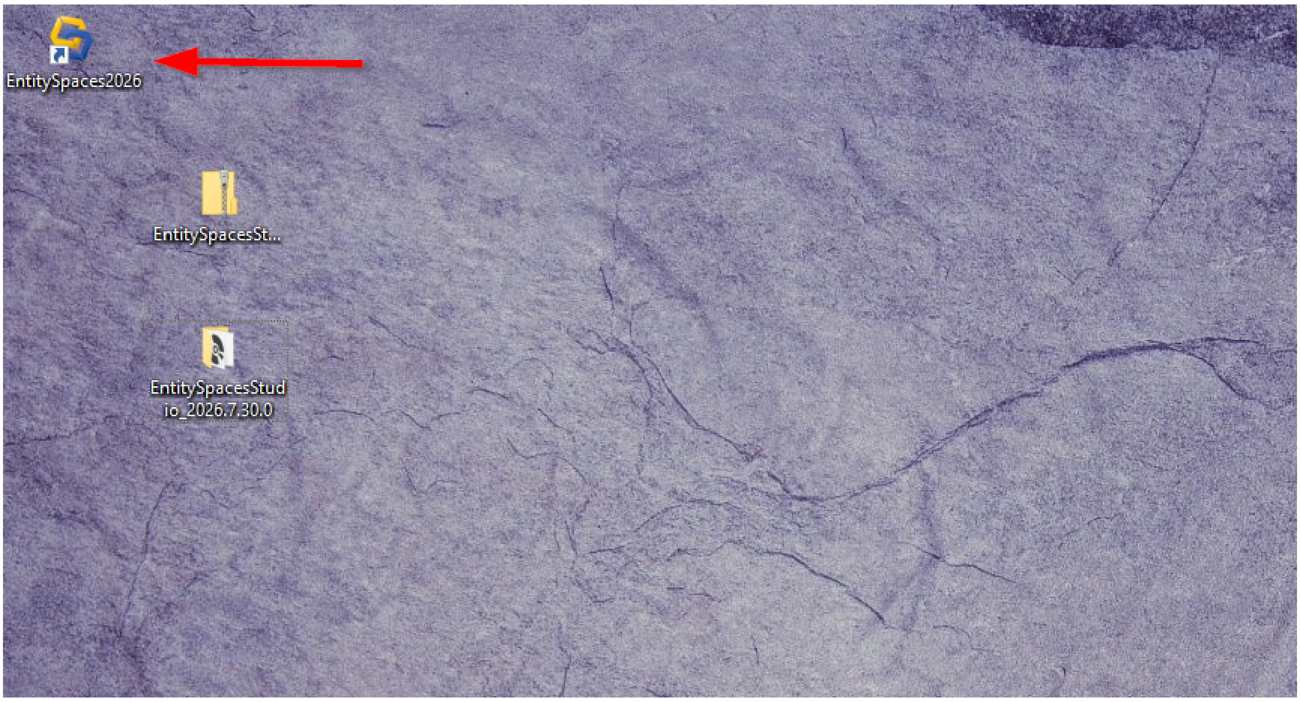


Once the installation is finished, check **"Launch EntitySpaces Studio"** if you want to open the Studio immediately. Click **"Finish"** to close the wizard.

---

3. First Launch and Configuration

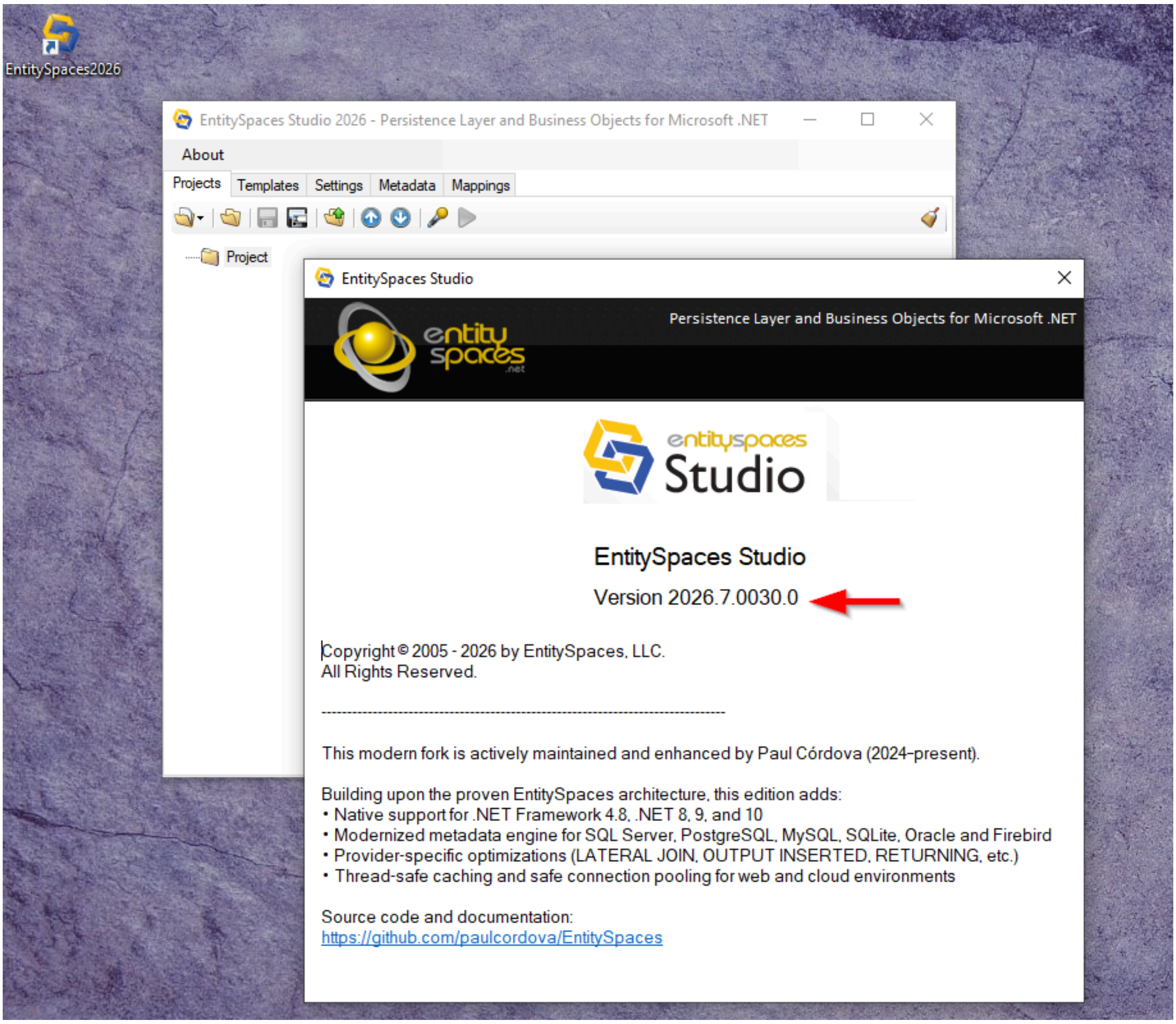
3.1. Access the Studio



You can open the Studio from the shortcut created on the **Desktop** or from the **Start Menu** → **EntitySpaces 2026**.

> ⚠️ **Important:** Run the Studio **as administrator** the first time (right-click → "Run as administrator"). This allows the Studio to create the configuration folders in `C:\ProgramData\EntitySpaces\ES2026` with the correct permissions for all users.

3.2. Verify the Installed Version



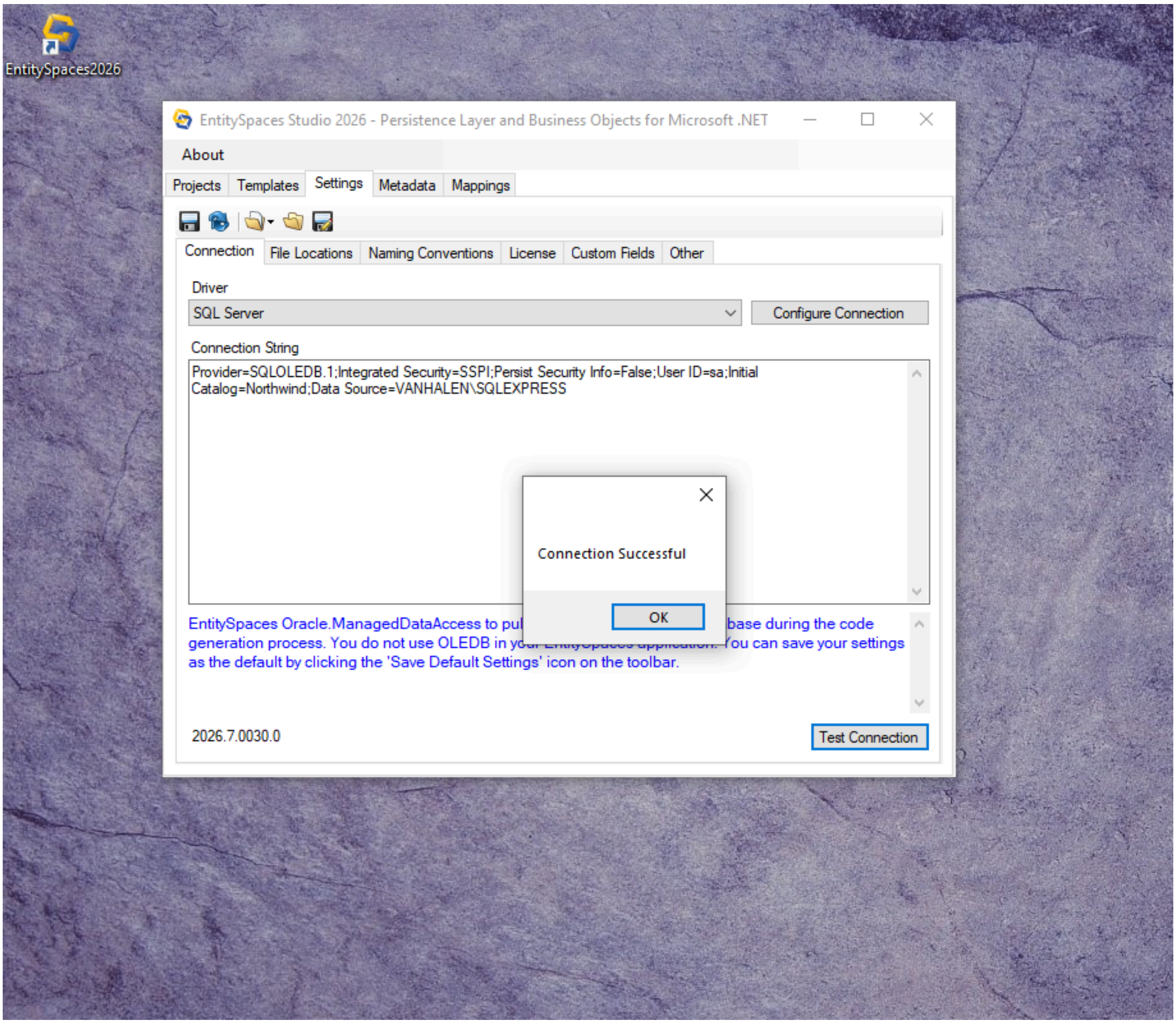
In the main window, go to **Help** → **About...** and confirm that the version is the expected one (e.g., `2026.7.0030.0`). This ensures the installation was successful.

---

4. Connect to Your Database

> ⚠ **Important:** This section is a prerequisite for code generation; you need an active connection so the Studio can read the schema.

1. Go to the **Settings** → **Connection** tab.
2. Select the **Driver** corresponding to your database engine (SQL Server, PostgreSQL, MySQL, SQLite, Oracle, Firebird).
3. Click **Configure Connection** to build the connection string, or type it manually.
4. Test the connection with the **Test Connection** button.



Example: Connection setup for SQL Server Local.

Sample connection strings:

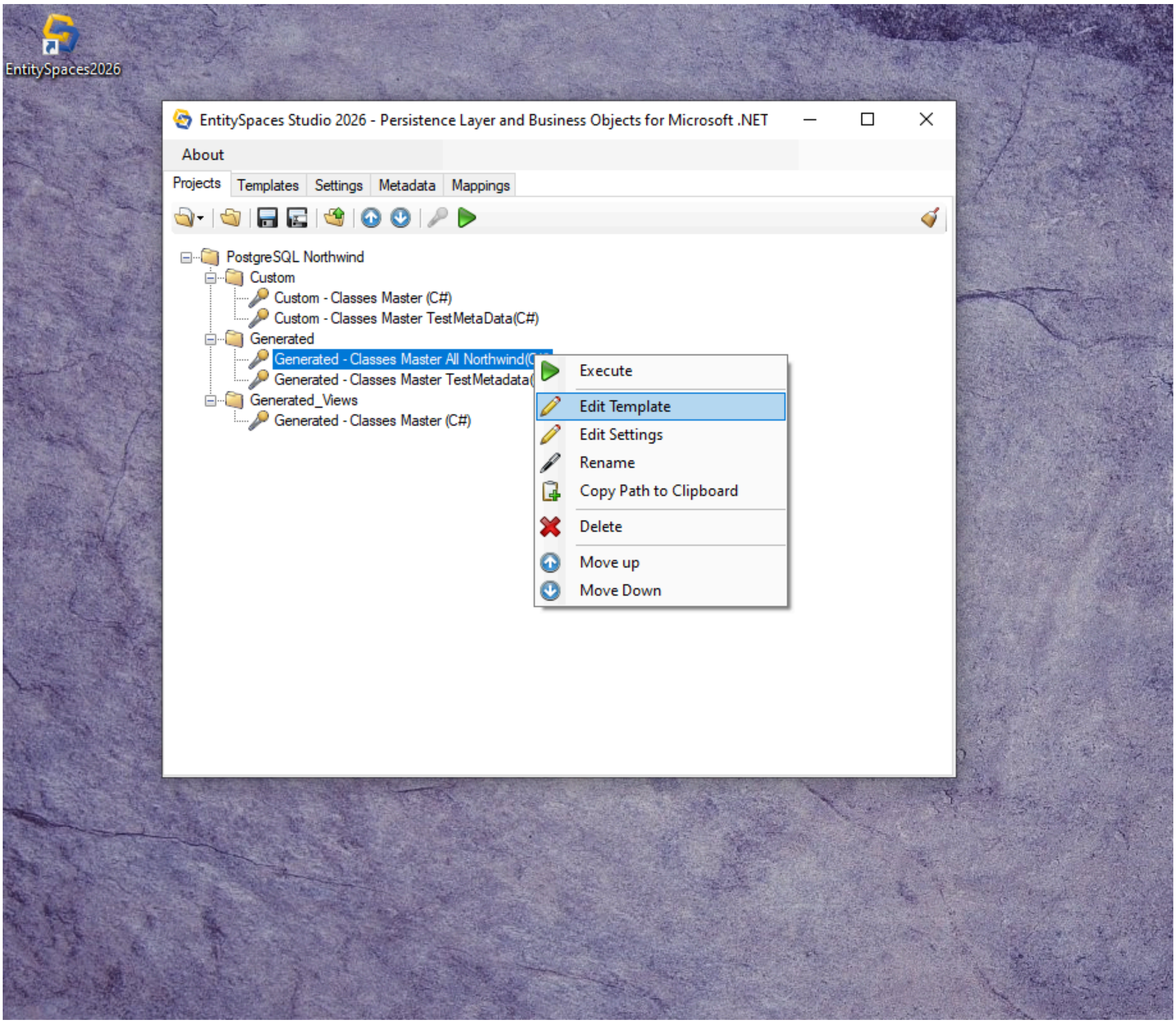
Engine	Example string
SQL Server	`User ID=sa;Password=...;Initial Catalog=Northwind;Data Source=localhost;TrustServerCertificate=True;`
PostgreSQL	`Host=localhost;Port=5432;Database=Northwind;Username=postgres;Password=...;SSL Mode=Require;`
MySQL	`Server=localhost;Port=3306;Database=Northwind;Uid=root;Pwd=...;SslMode=Required;`
SQLite	`Data Source=C:\Northwind.db3;Version=3;Foreign Keys=True;`
Oracle	`Data Source=...;User Id=...;Password=...;` (plus wallet configuration for ATP)

---

5. Generate C# or VB.NET Classes

5.1. Open the Templates Panel

In the left panel of the Studio, expand your project tree (or create a new one in **File → New Project**).
Right-click on the folder where you want to generate the code and select **"Add Recorded Template"** or **"Execute Template"**.



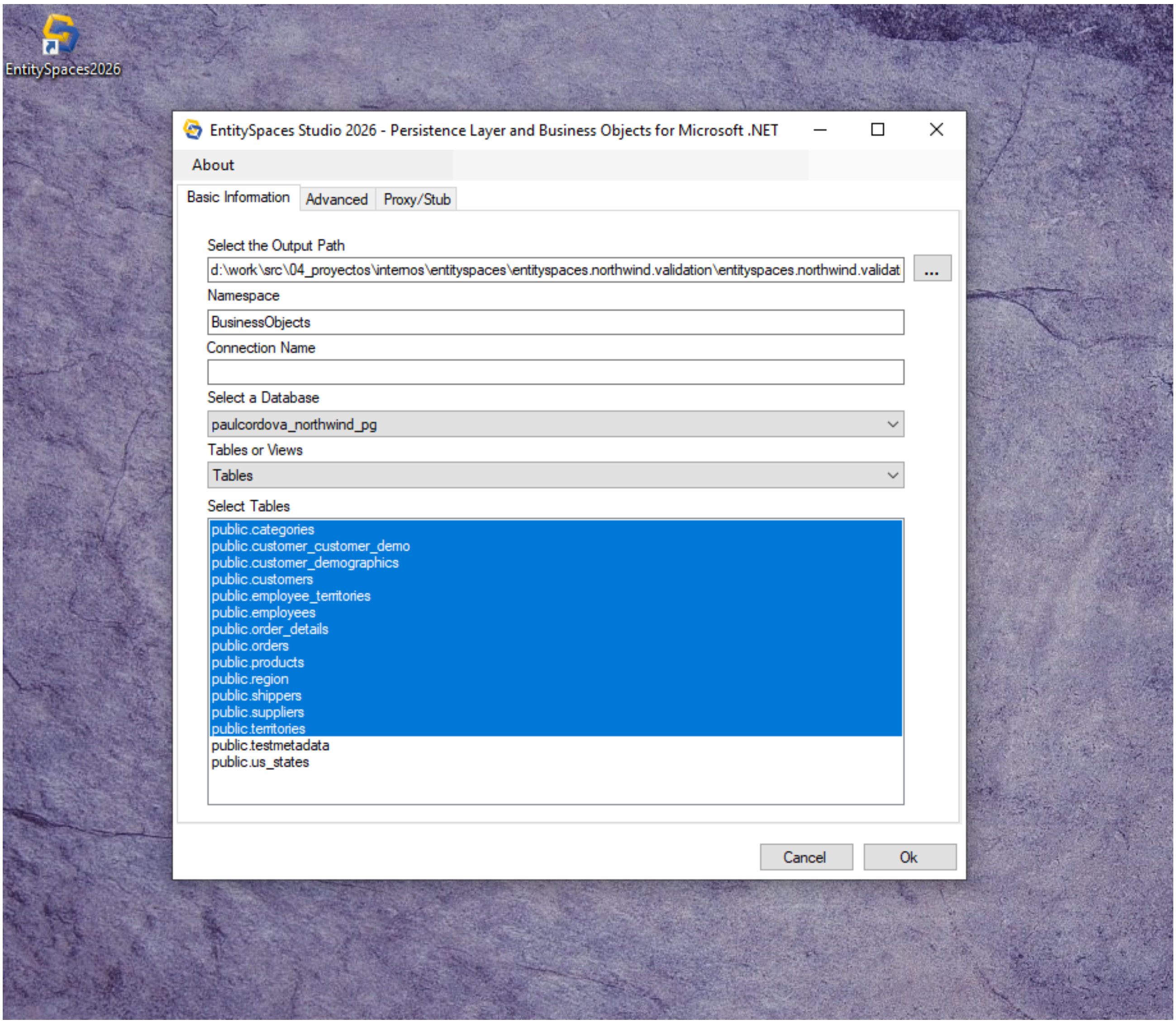
5.2. Select the "Generated - Classes Master" Template

The template selection dialog will appear. Navigate to the **C#** (or VB) folder and choose **"Generated - Classes Master"**. Click **"OK"**.

5.3. Configure Generation Parameters

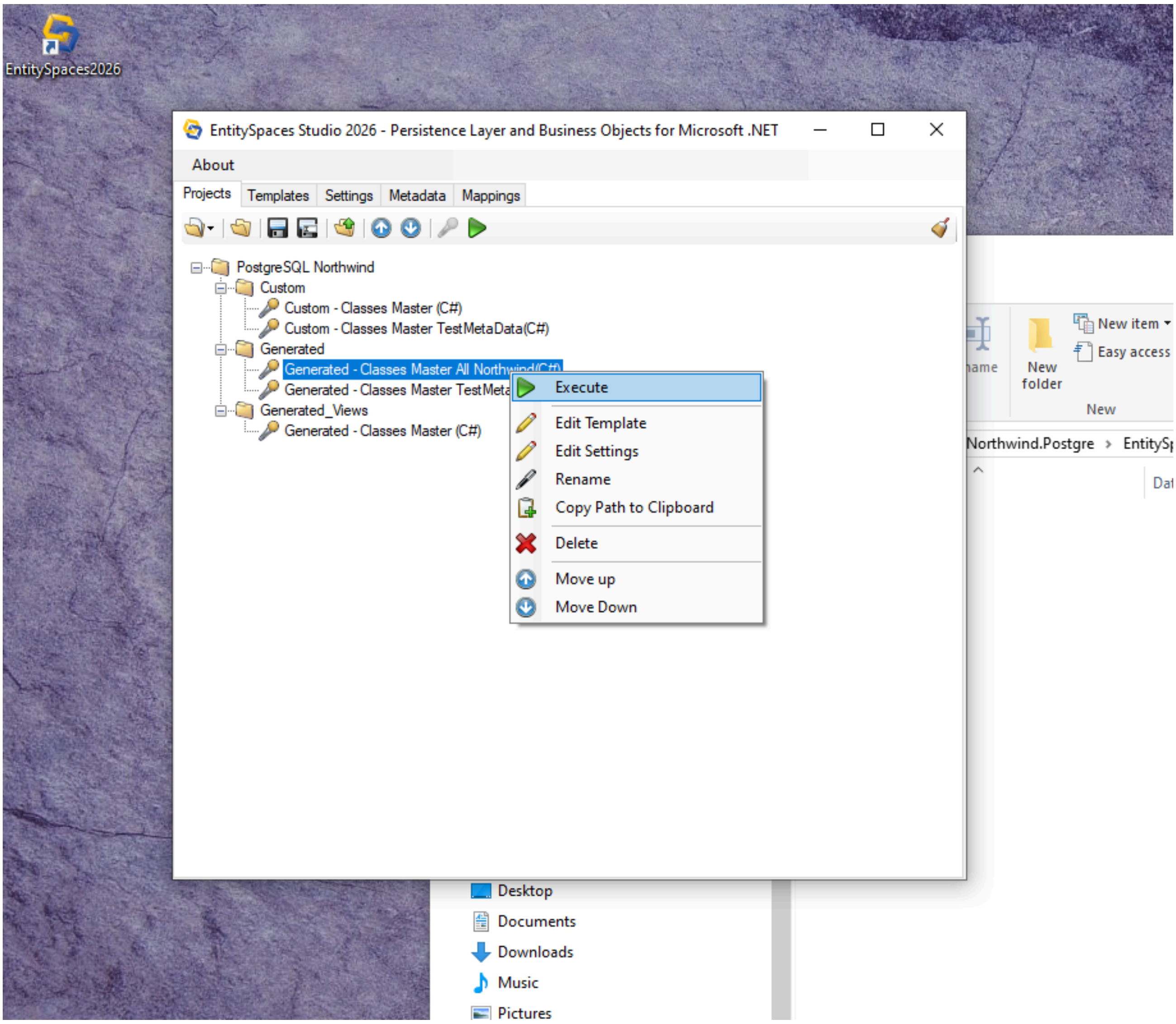
In the configuration window:

- Define the **Namespace** (e.g., `BusinessObjects`).
- In the **"Input"** tab, select the tables or views you want to generate.
- You can use the **"Select All"** button to choose all of them.
- In the **"Advanced"** tab, review options such as `Generate a Single File` (recommended) and `Metadata Class Should Ignore Schema` (to avoid including the schema name in properties).



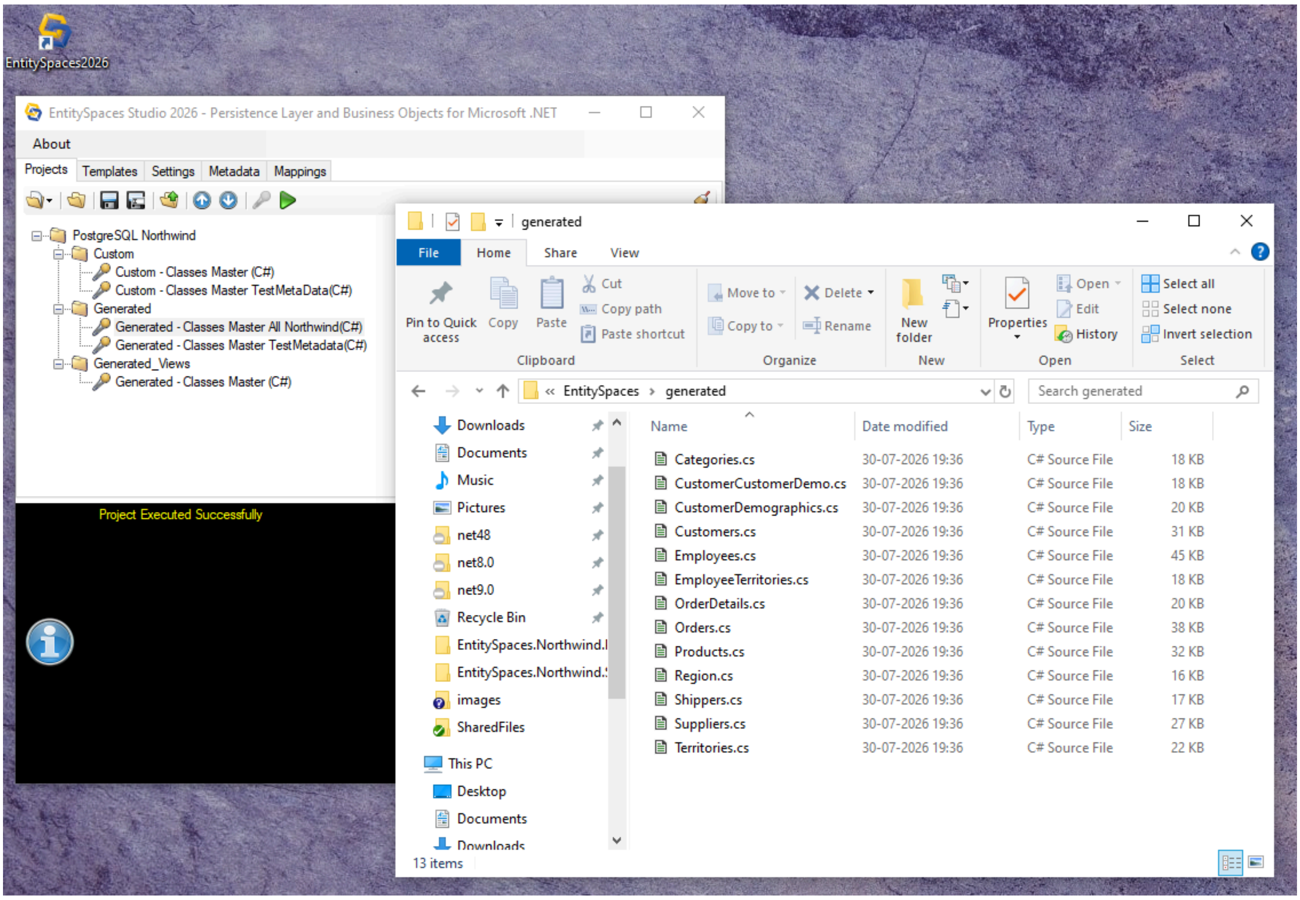
5.4. Run the Generation

Click **"OK"**. The Studio will generate the files in the specified output folder (by default, a `Generated` subfolder within your project folder).
You will see the progress at the bottom of the window.



5.5. Verify the Generated Code

In less than a minute, you will have all the files ready. You can open them with your preferred editor or directly from File Explorer.



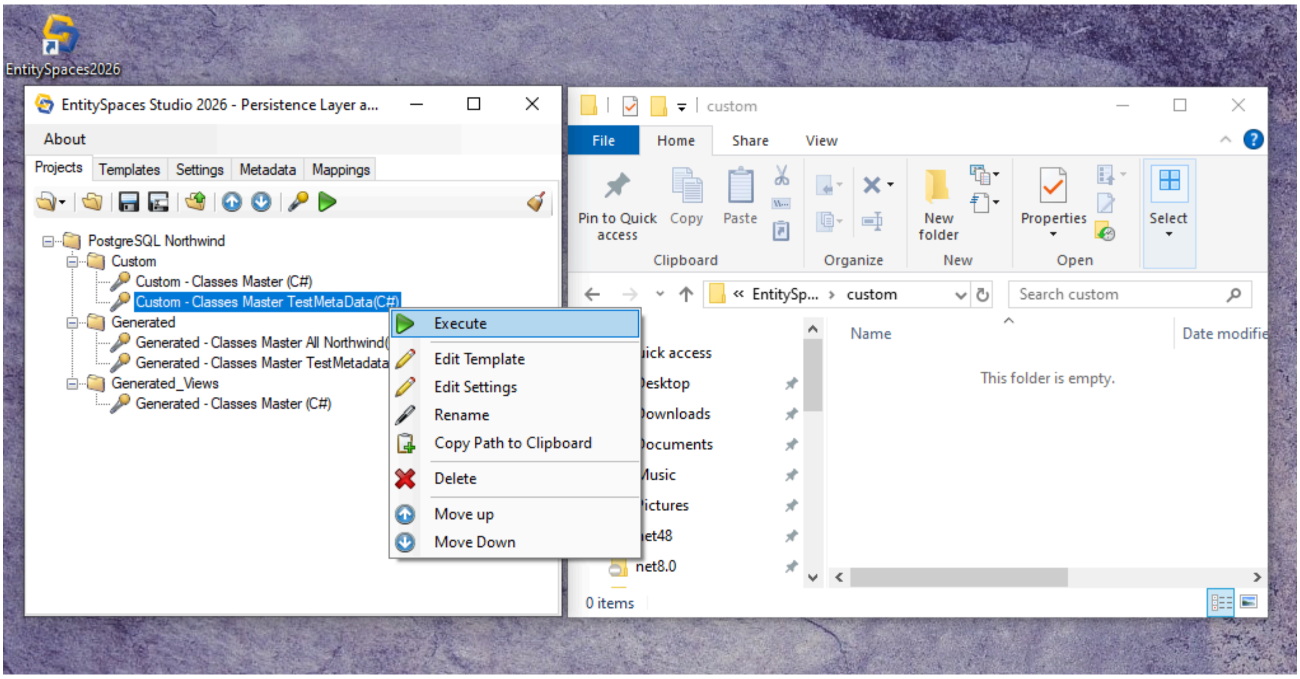
---

6. Generate Custom Classes (Generated Once)

The **Custom** classes are generated only once and are the place where you add your business logic, validations, or additional methods. **They are not overwritten** when regenerating.

6.1. Run the "Custom - Classes Master" Template

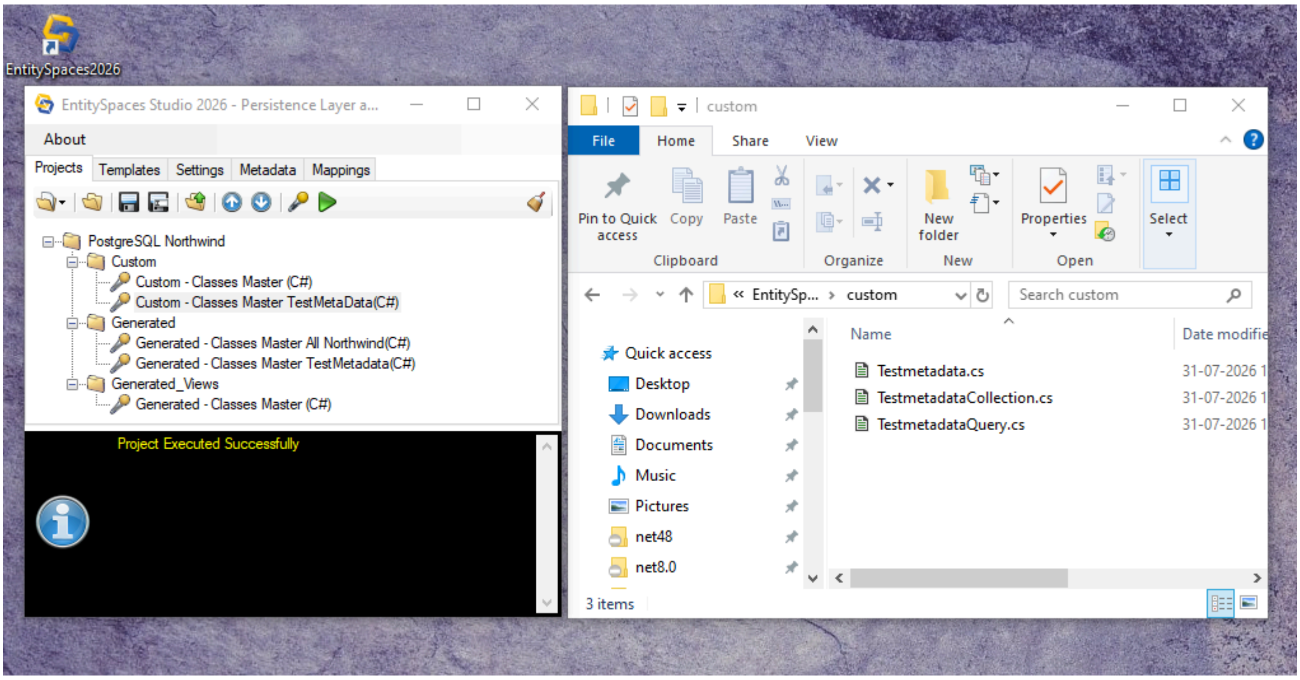
Follow the same process as in the previous step, but select the **"Custom - Classes Master"** template.



6.2. Review the Generated Structure

For each table, three classes are generated:

- `{TableName}` – the entity (Custom).
- `{TableName}Collection` – the collection (Custom).
- `{TableName}Query` – the fluent query (Custom).



6.3. Explore the Custom Class Code

You can freely open and modify these classes to add calculated properties, business methods, or overrides of `Save()`.

```csharp
/*
===============================================================================
                    EntitySpaces Studio by EntitySpaces, LLC
             Persistence Layer and Business Objects for Microsoft .NET
             EntitySpaces(TM) is a legal trademark of EntitySpaces, LLC
                          http://www.entityspaces.net
===============================================================================
EntitySpaces Version : 2026.7.0030.0
EntitySpaces Driver  : PostgreSQL
Date Generated       : 31-07-2026 12:19:45
===============================================================================
*/

using System;

using EntitySpaces.Core;
using EntitySpaces.Interfaces;
using EntitySpaces.DynamicQuery;

namespace BusinessObjects
{
    public partial class Testmetadata : esTestmetadata
    {
        public Testmetadata()
        {

        }
    }
}
```

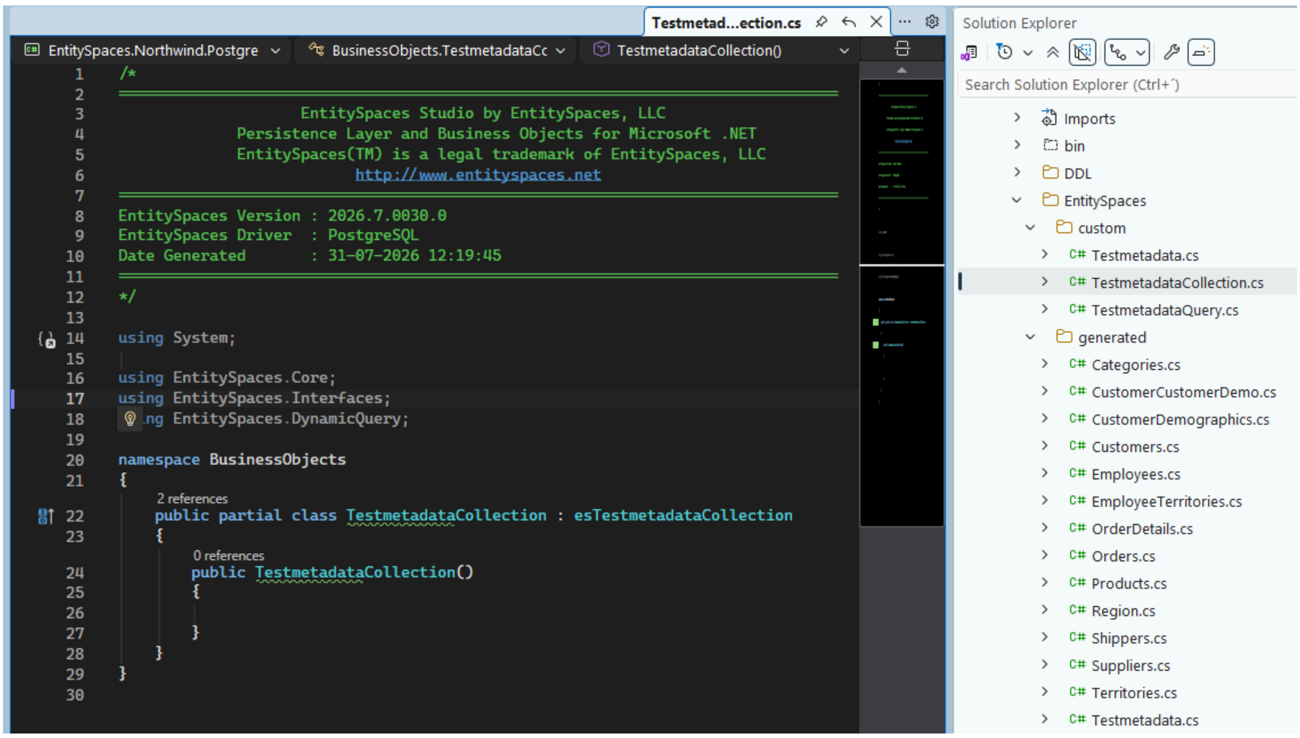
```csharp
/*
===============================================================================
                    EntitySpaces Studio by EntitySpaces, LLC
             Persistence Layer and Business Objects for Microsoft .NET
             EntitySpaces(TM) is a legal trademark of EntitySpaces, LLC
                          http://www.entityspaces.net
===============================================================================
EntitySpaces Version : 2026.7.0030.0
EntitySpaces Driver  : PostgreSQL
Date Generated       : 31-07-2026 12:19:45
===============================================================================
*/

using System;

using EntitySpaces.Core;
using EntitySpaces.Interfaces;
using EntitySpaces.DynamicQuery;

namespace BusinessObjects
{
    public partial class TestmetadataCollection : esTestmetadataCollection
    {
        public TestmetadataCollection()
        {

        }
    }
}
```

```csharp
/*
===============================================================================
                EntitySpaces Studio by EntitySpaces, LLC
         Persistence Layer and Business Objects for Microsoft .NET
         EntitySpaces(TM) is a legal trademark of EntitySpaces, LLC
                      http://www.entityspaces.net
===============================================================================
EntitySpaces Version : 2026.7.0030.0
EntitySpaces Driver  : PostgreSQL
Date Generated       : 31-07-2026 12:19:45
===============================================================================
*/

using System;

using EntitySpaces.Core;
using EntitySpaces.Interfaces;
using EntitySpaces.DynamicQuery;

namespace BusinessObjects
{
    public partial class TestmetadataQuery : esTestmetadataQuery
    {
        public TestmetadataQuery()
        {

        }
    }
}
```

---

7. Folder Structure in Visual Studio

Once the files are generated, add them to your Visual Studio project. The recommended structure is to separate the `Generated` classes (which are regenerated) from the `Custom` classes (which contain your logic).



- `Generated` **folder**: Contains automatically generated files. **Do not edit them** (they are overwritten on regeneration).
- `Custom` **folder**: Contains your business classes. **This is where you work**.

8. Unit Testing with EntitySpaces and PostgreSQL

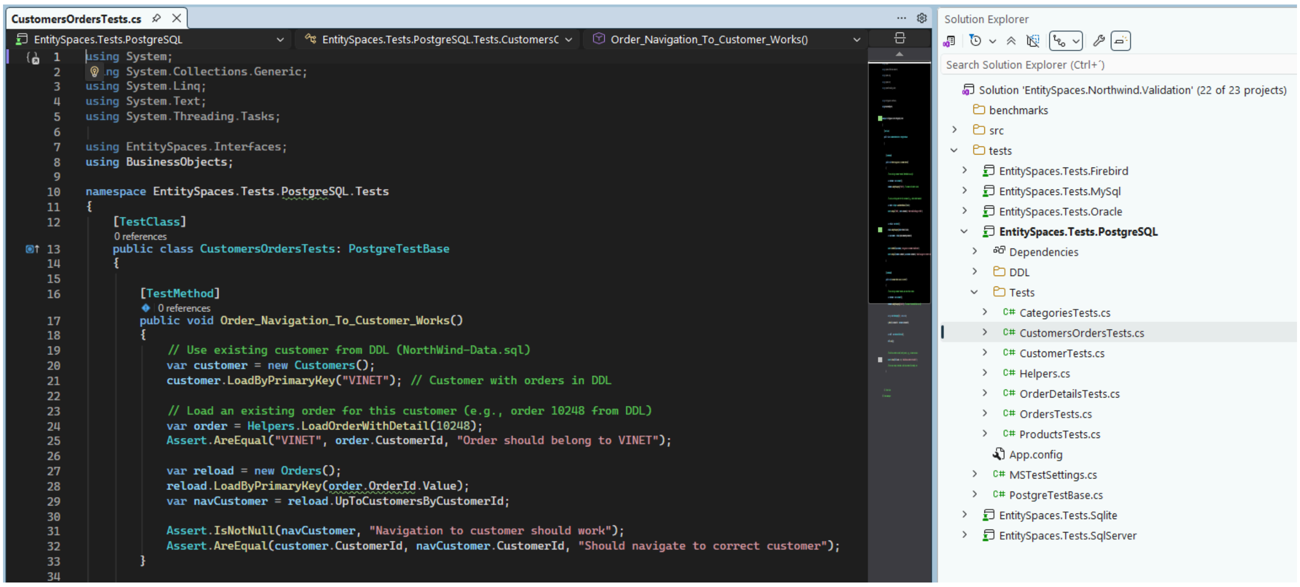
To verify that your configuration and classes work correctly, you can write simple unit tests.

8.1. Configure the Loader and Connection in Your Test

```
[SetUp]
public void Setup()
{
    esProviderFactory.Factory = new EntitySpaces.LoaderMT.esDataProviderFactory();
    esConnectionElement conn = new esConnectionElement();
    conn.Provider = "EntitySpaces.PostgreSQLProvider";
    conn.ConnectionString =
"Host=localhost;Port=5432;Database=Northwind;Username=postgres;Password=...;SSL
Mode=Require;";
    esConfigSettings.ConnectionInfo.Connections.Add(conn);
    esConfigSettings.ConnectionInfo.Default = "Northwind";
}
```

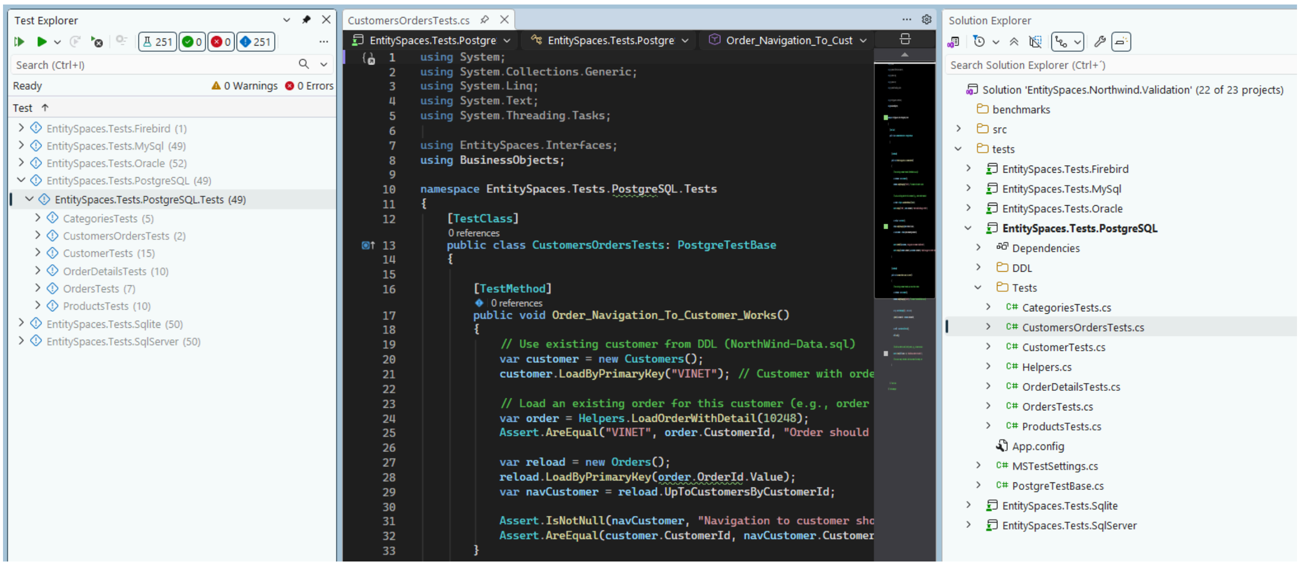
8.2. Load and Query Tests

```
[Test]
public void CanLoadAllCustomers()
{
    var customers = new CustomersCollection();
    Assert.IsTrue(customers.LoadAll());
    Assert.Greater(customers.Count, 0);
}

[Test]
public void CanQueryCustomersByCountry()
{
    var coll = new CustomersQuery("c", out var c)
        .Where(c.Country == "USA")
        .ToCollection<CustomersCollection>();
    Assert.Greater(coll.Count, 0);
}
```

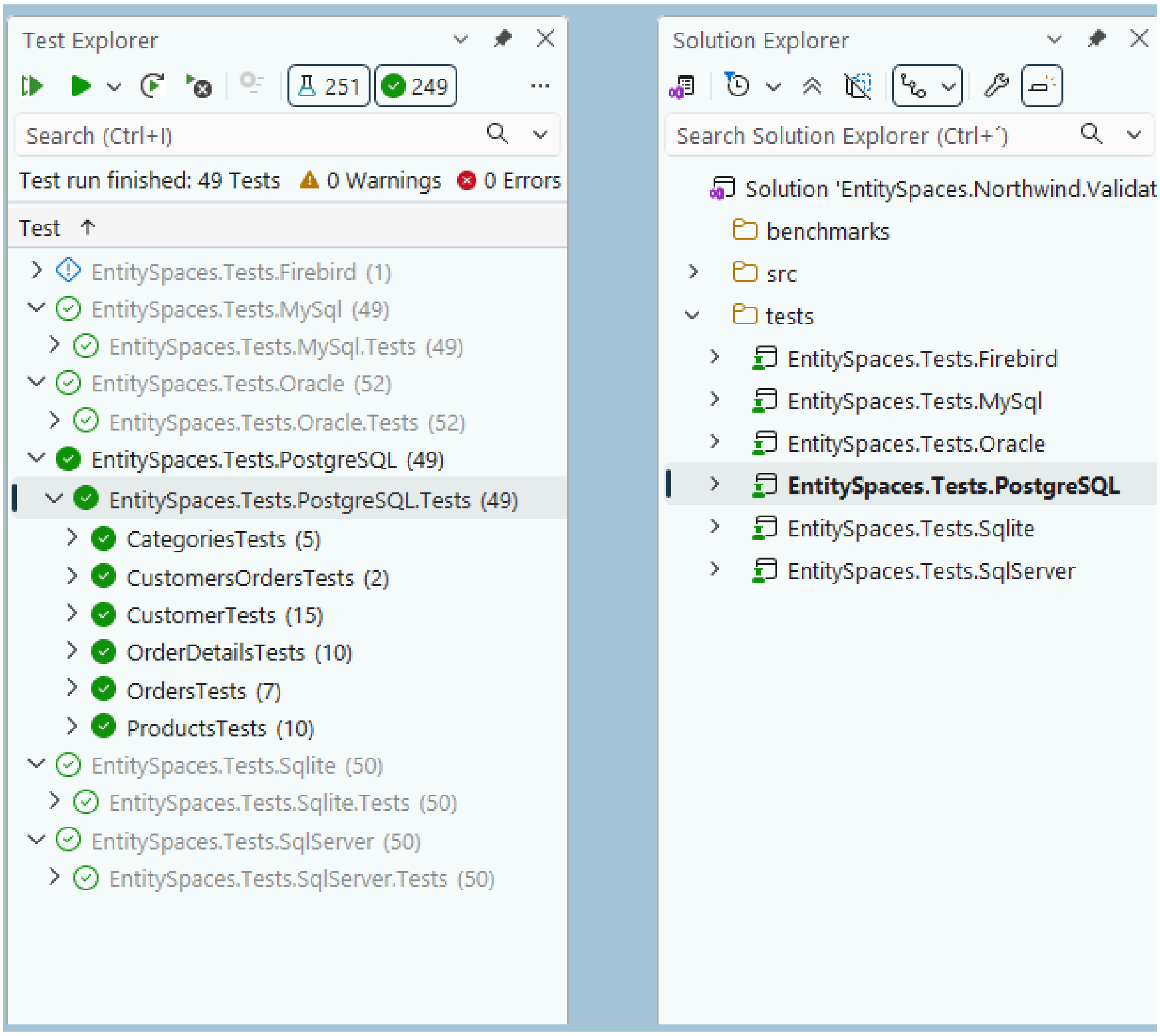


8.3. Run the Tests

Make sure the PostgreSQL provider is correctly referenced (NuGet package `EntitySpaces.ORM.PostgreSQL.NET`).



If everything is correct, the tests will run successfully.



---

9. Add NuGet Packages to Your Project

Open your project in Visual Studio and add the package for the provider you need.

From the Package Manager Console:

```
# Example for SQL Server
Install-Package EntitySpaces.ORM.SqlServer.NET

# For PostgreSQL
Install-Package EntitySpaces.ORM.PostgreSQL.NET

# For MySQL
Install-Package EntitySpaces.ORM.MySQL.NET

# For SQLite
Install-Package EntitySpaces.ORM.SQLite.NET

# For Oracle Managed Client
Install-Package EntitySpaces.ORM.OracleManagedClient.NET
```

```
# For Firebird (coming soon)
Install-Package EntitySpaces.ORM.Firebird.NET
```

These packages include all necessary dependencies (`EntitySpaces.Core`, `EntitySpaces.Interfaces`, `EntitySpaces.DynamicQuery`, the Loader, and the specific provider).

---

10. Configure the Loader and Connection in Your Code

EntitySpaces requires you to register the **Loader** (medium trust or standard) and the **connection string**.

At the start of your application (e.g., `Program.Main`, `Global.asax`, or the main form constructor):

```
using EntitySpaces.Interfaces;

// 1. Register the Loader (medium trust version, recommended)
esProviderFactory.Factory = new EntitySpaces.LoaderMT.esDataProviderFactory();

// 2. Configure the connection
esConnectionElement conn = new esConnectionElement();
conn.Provider = "EntitySpaces.SqlClientProvider";   // Change according to your
provider
conn.ConnectionString = "User ID=...;Password=...;Initial Catalog=Northwind;Data
Source=localhost;";
conn.SqlAccessType = esSqlAccessType.DynamicSQL;    // or StoredProcedure
conn.DatabaseVersion = "2016";                      // Optional, auto-detected

// 3. Add the connection and set it as default
esConfigSettings.ConnectionInfo.Connections.Add(conn);
esConfigSettings.ConnectionInfo.Default = "Northwind"; // name of your choice
```

Alternative: use an `app.config` / `web.config` file with the `<EntitySpaces>` section. The complete guide is available in the repository's README.

---

11. Write Your First Query

With the generated classes and configuration ready, you can load data:

```
// Load all customers
CustomersCollection customers = new CustomersCollection();
if (customers.LoadAll())
{
    foreach (Customers cust in customers)
    {
        Console.WriteLine($"{cust.CustomerID} - {cust.CompanyName}");
    }
}

// Query with filter (fluent API)
CustomersCollection coll = new CustomersQuery("c", out var c)
    .Where(c.Country == "USA")
    .OrderBy(c.CompanyName.Ascending)
```

```
    .ToCollection<CustomersCollection>();

// Basic CRUD
Customers newCust = new Customers();
newCust.CustomerID = "NEWCO";
newCust.CompanyName = "New Company";
newCust.Save();                      // INSERT

newCust.CompanyName = "New Company Inc.";
newCust.Save();                      // UPDATE

newCust.MarkAsDeleted();
newCust.Save();                      // DELETE
```

---

12. Using with Wisej.NET

Wisej.NET is a framework that lets you build web applications with the same logic and event model as Windows Forms. EntitySpaces integrates seamlessly:

- You can reuse all your generated classes and existing business logic.
- The connection and transaction patterns are identical to those of a desktop application.
- The ORM correctly handles pooling and concurrency in the web environment (thanks to thread-safe parameter caching and transaction handling in `finally` blocks).

Example in a Wisej control:

```
// In a button Click event
private void buttonLoad_Click(object sender, EventArgs e)
{
    var products = new ProductsCollection();
    products.Query.Where(products.Query.UnitPrice > 10);
    products.Query.Load();

    dataGridView1.DataSource = products;
}
```

Refer to the official Wisej.NET documentation for more details on deployment.

---

13. Common Troubleshooting

Problem	Solution
"Assembly EntitySpaces.Core not found"	Make sure you installed the correct NuGet package and that it is referenced in your project.
"Error connecting to SQLite"	Verify that the connection string includes `Version=3;Foreign Keys=True;` and that the `.db3` file exists.
Studio does not see tables	In the Studio, ensure you have selected the correct catalog/database in the dropdown after connecting.

Problem	Solution
Concurrency exception when saving	EntitySpaces throws `esConcurrencyException` for key violations, deadlocks, etc. Catch it and handle the conflict according to your business logic.
Permission error in `ProgramData\EntitySpaces\ES2026`	Make sure to run the Studio as administrator the first time, or grant write permissions to the folder for your user (or the "Authenticated Users" group).
Studio does not save the MRU project list	Verify that the registry key `HKEY_CURRENT_USER\Software\EntitySpaces 2026` exists and that your user has write permissions. If it does not exist, run the Studio as administrator once to create it.

---

14. Additional Resources

- **GitHub Repository:** https://github.com/paulcordova/EntitySpaces
  - Source code, full examples, change history.
- **Extended documentation:** The `README.md` file in the repository contains all technical details, including metadata migration, providers, and best practices.
- **Forum / Issues:** Use GitHub Issues to report problems or suggest improvements.
- **Consulting:** Contact `paul.cordova@netstep.cl` for professional support or training.

Ready! With these steps, your environment is up and running, and you can start developing with EntitySpaces 2026.